

Análisis FAPI 2.0 para Finanzas Abiertas (Chile) y soporte en Keycloak

Documento técnico que evalúa los 18 requisitos de seguridad exigibles al *Authorization Server* (Identity Server) bajo FAPI 2.0, en el contexto del **Sistema de Finanzas Abiertas (SFA)** de la **CMF de Chile** (NCG N.º 514 y su normativa técnica complementaria, basada en el perfil **FAPI 2.0 Security Profile + Message Signing** de la OpenID Foundation).

Para cada ítem se entrega:

1. Contexto técnico (definición, diagrama, ejemplos).
2. Obligatoriedad en FAPI 2.0 / SFA Chile.
3. Soporte nativo en **Keycloak ≥ 26.x**.
4. Recomendaciones de implementación si no está soportado.

Al final se enumeran los **aspectos relevantes que no aparecen en la lista original** y que también son exigibles para certificarse en FAPI 2.0.

Tabla de Acrónimos

Sigla	Significado
FAPI	Financial-grade API (perfil de seguridad OAuth/OIDC de la OpenID Foundation)
SFA	Sistema de Finanzas Abiertas (Chile)
CMF	Comisión para el Mercado Financiero
SSA	Software Statement Assertion (RFC 7591)
DCR	Dynamic Client Registration (RFC 7591/7592)
PAR	Pushed Authorization Requests (RFC 9126)
JAR	JWT-Secured Authorization Request (RFC 9101)
JARM	JWT Secured Authorization Response Mode (OIDF)
PKCE	Proof Key for Code Exchange (RFC 7636)
DPoP	Demonstrating Proof of Possession (RFC 9449)
mTLS	Mutual TLS for OAuth (RFC 8705)
RAR	Rich Authorization Requests (RFC 9396)
OCSP	Online Certificate Status Protocol (RFC 6960)
SPI	Service Provider Interface (mecanismo de extensión de Keycloak)

Resumen ejecutivo – Matriz de cumplimiento

#	Requisito	¿Obligatorio FAPI 2.0?	Soporte Keycloak 26.x
1	Compatible con FAPI 2.0	Sí	✓ Parcial (Security Profile, Message Signing en preview)
2	mTLS 1.3	Sí (mTLS), TLS 1.3 recomendado	✓ Soportado (depende del <i>reverse-proxy</i>)
3	ES256 / PS256	Sí	✓ Soportado
4	Rotación de Refresh Tokens	Sí (o sender-constrained)	✓ Soportado
5	Deshabilitar flujos legacy	Sí	✓ Soportado vía <i>Client Policies</i>
6	Sender-Constrained Tokens	Sí (mTLS o DPoP)	✓ Soportado
7	DPoP	Alternativa obligatoria	✓ GA desde 26.x
8	DCR	Sí (Open Finance)	✓ Parcial (sin validación de SSA out-of-the-box)
9	Consumo de SSA	Sí (SFA Chile)	✗ Requiere SPI
10	JARM	Sí en <i>Message Signing</i>	✓ Soportado
11	PAR obligatorio	Sí	✓ Soportado
12	PKCE obligatorio	Sí (S256)	✓ Soportado
13	OCSP	Sí (cadena PKI)	⚠ Parcial
14	OCSP Stapling	Recomendado	⚠ Vía reverse-proxy
15	Consentimiento granular	Sí (SFA)	⚠ Parcial
16	RAR (RFC 9396)	Opcional en FAPI 2.0, exigido por SFA	⚠ Parcial (claim <code>authorization_details</code> configurable)
17	Mapeo de scopes ↔ recursos legales	Sí (SFA)	✗ Requiere extensión
18	Logs criptográficos no-repudiables	Sí (SFA, trazabilidad)	✗ Requiere extensión

Leyenda: ✓ soporte nativo, ⚠ soporte parcial, ✗ no soportado nativamente.

1. Compatible con FAPI 2.0

1.1 Contexto técnico

FAPI 2.0 es la evolución de FAPI 1.0 publicada por la **OpenID Foundation Financial-grade API Working Group**. Está compuesto por dos perfiles formales:

- **FAPI 2.0 Security Profile** (ID-2 / Final): conjunto **base** de controles que el AS y el cliente deben cumplir.

- **FAPI 2.0 Message Signing:** capa adicional **sobre** el Security Profile que exige firma JWS de mensajes (request object + JARM) para escenarios de no-repudio (típicamente *write/payment*).

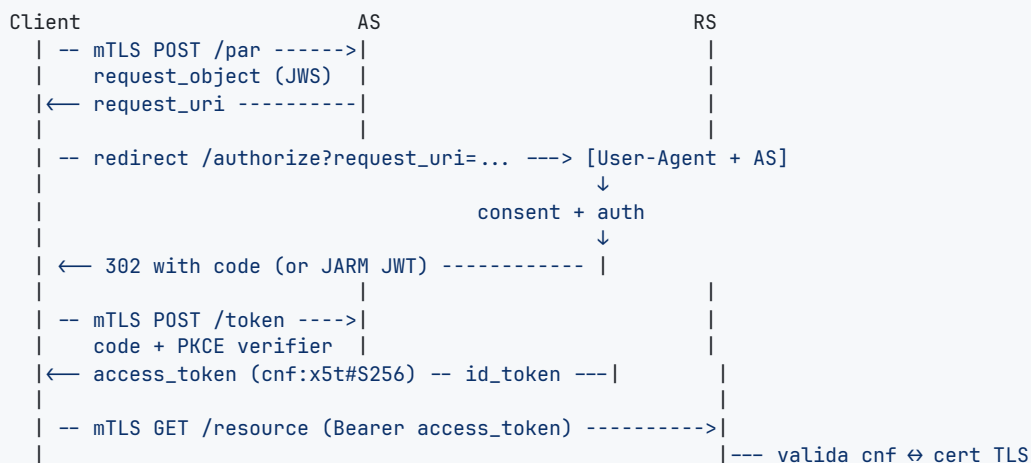
Se construyó como simplificación del FAPI 1.0 Advanced eliminando ambigüedades, removiendo `response_type=code id_token`, exigiendo PAR siempre y eliminando `s_hash`.

Modelo de amenazas cubierto

FAPI 2.0 está formalmente verificado contra el **Attacker Model** definido por OIDF, cubriendo:

- Atacante de red (incluido TLS-MITM con certificado válido para otra entidad).
- Atacante con control de un endpoint del cliente.
- Atacante con acceso al *user agent*.
- Cross-site request injection.
- Mix-up attacks (mediante `iss` en la respuesta — RFC 9207).

Diagrama (flujo "code with PAR + mTLS")



1.2 ¿Obligatorio?

Sí. La NCG 514 de la CMF y la "Norma Técnica de Seguridad del SFA" establecen que la autenticación y autorización **debe** seguir FAPI 2.0 Security Profile, y en operaciones con efectos financieros (movimiento de fondos, iniciación de pagos) **FAPI 2.0 Message Signing**.

1.3 Soporte Keycloak

✅ **Parcial.** Keycloak incorpora desde la versión 22.x un *Client Policy Profile* llamado `fapi-2-security-profile` y `fapi-2-message-signing`. Activarlo:

```
kc.sh start --features=fapi-2,par,dpop,token-exchange
```

Y en la realm:

```
Realm Settings → Client Policies → Profiles → fapi-2-security-profile
                  → Policies → "FAPI 2.0 clients" (matcher por client-attr)
```

Limitaciones conocidas (a la fecha):

- El profile no fuerza automáticamente `iss` en la respuesta de autorización (RFC 9207); está, pero hay que validar la versión.
- El profile de Message Signing es Tech-Preview en algunas releases LTS, GA en 26.x.

1.4 Recomendaciones

- Mantener Keycloak en **26.x LTS** o superior.
- Validar la conformidad ejecutando la **OpenID Foundation Conformance Suite** (`fapi2-security-profile-id2-test-plan`).
- Documentar internamente el *gap* respecto al perfil chileno (por ejemplo, requisitos puntuales del SFA que sean estrictamente mayores que el FAPI 2.0 base).

2. Soporte mTLS 1.3

2.1 Contexto técnico

Mutual TLS es la autenticación **bidireccional** a nivel TLS: además de validar el certificado del servidor, el cliente presenta un certificado X.509 firmado por una CA confiable.

En el ecosistema FAPI/Open Finance se usa para:

1. **Autenticación del cliente OAuth** (`tls_client_auth` o `self_signed_tls_client_auth`, RFC 8705 §2).
2. **Sender-Constrained Access Tokens**: el AS calcula `cnf.x5t#S256` (hash SHA-256 del cert) y el RS exige que el token solo sea usable sobre una conexión TLS donde el cliente presenta ese mismo cert (RFC 8705 §3).

TLS 1.3 (RFC 8446) elimina algoritmos débiles (RC4, SHA-1, MD5, CBC, RSA-KEX, renegociación) y mTLS sobre TLS 1.3 se solicita mediante `post_handshake_auth`.

Diagrama



2.2 ¿Obligatorio?

Sí. FAPI 2.0 exige *sender-constrained access tokens* y la única opción además de DPoP es mTLS. TLS 1.3 no es estrictamente exigido por la RFC pero la CMF requiere **TLS 1.2 mínimo, TLS 1.3 recomendado**; varias

guías chilenas marcan TLS 1.3 como obligatorio para 2026+.

2.3 Soporte Keycloak

✓ Soportado. Keycloak corre sobre **Quarkus + Netty** y soporta TLS 1.3 (Java 17/21). El *handshake* mTLS puede:

- Terminarse en el propio Keycloak (`https-client-auth=request|required`).
- Terminarse en un **reverse-proxy** (NGINX/HAProxy/Envoy) que reenvía el certificado por `X-SSL-Client-Cert` → habilitar `spi-x509cert-lookup-provider=nginx|haproxy|apache` .

```
# Terminación directa
https-client-auth=request
https-trust-store-file=/etc/keycloak/truststore.p12
https-trust-store-password=...

# Terminación en NGINX
spi-x509cert-lookup-provider=nginx
```

En el cliente OAuth (Realm → Clients → Credentials):

- Client Authenticator: `X.509 Certificate (tls_client_auth)` o `Self-signed Certificate (self_signed_tls_client_auth)` .

2.4 Recomendaciones si no soportado

No aplica (sí está soportado). Buenas prácticas:

- Forzar TLS 1.3 only en el `listen` del *edge*.
- Suite cifrada: `TLS_AES_128_GCM_SHA256` , `TLS_AES_256_GCM_SHA384` , `TLS_CHACHA20_POLY1305_SHA256` .
- Habilitar `ssl_verify_client on` y `ssl_verify_depth ≥ 2` (sub-CA del SFA).

3. Algoritmos de firma ES256 / PS256

3.1 Contexto técnico

FAPI 2.0 restringe los algoritmos JWS válidos a:

- **PS256** – RSASSA-PSS con SHA-256 y MGF1.
- **ES256** – ECDSA con curva P-256 y SHA-256.
- (Opcionalmente **EdDSA** Ed25519 en el último draft.)

Se prohíben: `none` , `HS*` (HMAC – simétrico), `RS256` (RSA-PKCS#1 v1.5 – maleable).

Los componentes que **deben** firmarse con estos algoritmos son: `request_object` (JAR), `client_assertion` (`private_key_jwt`), `id_token` , `JARM response` , `software_statement` (SSA), `DPoP proof` .

3.2 ¿Obligatorio?

Sí. FAPI 2.0 Security Profile §5.3.1 (Authorization Server) enumera los valores admisibles para `id_token_signing_alg_values_supported`.

3.3 Soporte Keycloak

✓ Soportado. Se configura por cliente:

```
Clients → <cliente> → Advanced → Fine Grain OpenID Connect Configuration
- ID Token Signature Algorithm: PS256 / ES256
- User Info Signed Response Alg: PS256
- Request Object Signature Algorithm: PS256
- Token Endpoint Auth Signing Alg: PS256
```

A nivel de Realm: Realm Settings → Keys → generar key providers `rsa-enc-generated` (PS256) y `ecdsa-generated` (P-256, alg=ES256).

El profile `fapi-2-security-profile` ya **fuerza** estas restricciones por *Executor* (`secure-signature-algorithm-executor`).

3.4 Recomendaciones

- Usar Hardware Security Modules (HSM) vía PKCS#11 (`KC_DB_KEYSTORE_TYPE=pkcs11`) para custodia de la clave privada que firma `id_token` y JARM.
- Rotar las keys cada 12 meses como máximo (Realm Settings → Keys → Active key → priority, marcar la anterior como `passive`).

4. Rotación de Refresh Tokens

4.1 Contexto técnico

Refresh Token Rotation (RTR): cada vez que un cliente intercambia un `refresh_token` por un nuevo `access_token`, el AS emite también un **nuevo** `refresh_token` e **invalida** el anterior. Si el anterior se reutiliza, el AS asume **token theft** y revoca toda la familia (reuse detection).

```
RT1 —/token grant=refresh_token→ AS → AT2 + RT2 (RT1 invalido)
RT1 (reusado por atacante) → AS → 400 invalid_grant + REVOCAR familia
```

Es alternativa al modelo *sender-constrained refresh token* (mTLS/DPoP-bound).

4.2 ¿Obligatorio?

Sí, **condicional**. FAPI 2.0 §5.3.2.2 exige que los `refresh_token`:

MUST be sender-constrained using either mTLS or DPOP, OR MUST be rotated on each use.

En SFA Chile lo correcto es **ambos**: sender-constrained **y** rotación (defensa en profundidad).

4.3 Soporte Keycloak

✓ Soportado de forma nativa.

```
Realm Settings → Tokens
☑ Revoke Refresh Token      (activa la rotación + reuse detection)
Refresh Token Max Reuse: 0
SSO Session Idle: 30 min
Client Session Max: 1 hour
```

4.4 Recomendaciones

- Mantener Refresh Token Max Reuse = 0 .
- Definir Refresh Token Lifetime ≤ duración de consentimiento (en SFA: 180 días con re-confirmación a 90).
- Auditar evento REFRESH_TOKEN_ERROR para detectar reusos sospechosos (alimentar SIEM).

5. Inhabilitar flujos OAuth2 legacy

5.1 Contexto técnico

Flujos a **prohibir**:

Flow	Por qué
Resource Owner Password Credentials (ROPC)	Cliente maneja la contraseña ⇒ phishing, sin MFA, sin federación.
Implicit (response_type=token)	Token expuesto en URL fragment, sin PKCE, sin sender-constraint, no replay-proof.
Hybrid (code id_token , code token)	No usado por FAPI 2.0 (sí FAPI 1.0); aún válido sólo si Message Signing y JAR estricto, FAPI 2.0 lo elimina.
Authorization Code sin PKCE	Vulnerable a robo de code en clientes públicos.
Device Code (para clientes web)	No es el caso de uso financiero.

FAPI 2.0 únicamente permite **Authorization Code + PKCE + PAR**, y opcionalmente **CIBA** para flujos *decoupled*.

5.2 ¿Obligatorio?

Sí.

5.3 Soporte Keycloak

✓ Soportado.

Vía *Client Policies* (`fapi-2-security-profile` ya lo aplica) y manualmente:

```
Clients → <cliente> → Settings
Standard Flow Enabled      : ON   (auth code)
Direct Access Grants Enabled : OFF (deshabilita ROPC)
Implicit Flow Enabled      : OFF
Service Accounts Enabled   : ON   (solo para client_credentials machine-to-machine)
```

Y un **Executor** del profile:

```
secure-response-type-executor:
  allowed-response-types: ["code"]
secure-grant-types-executor:
  allowed-grant-types: ["authorization_code", "refresh_token", "client_credentials"]
```

5.4 Recomendaciones

- Aplicar las *Client Policies* globalmente por defecto (matcher `any-client`).
- Auditar al menos mensualmente el listado de *clients* para detectar habilitación accidental de Implicit/ROPC.

6. Sender-Constrained Tokens

6.1 Contexto técnico

Un *access_token* tradicional ("bearer") es portador: cualquiera que lo posea lo usa. Un token **sender-constrained** sólo es válido si el llamante demuestra poseer una **clave privada** asociada al token (Proof-of-Possession).

Dos mecanismos estandarizados:

1. **mTLS-bound** (RFC 8705): el AS embebe `cnf: { "x5t#S256": "<hash-cert>" }` en el token. El RS exige que la conexión TLS use el mismo cert.
2. **DPoP-bound** (RFC 9449): el AS embebe `cnf: { "jkt": "<jwk-thumbprint>" }` . El cliente envía en cada request un header `DPoP: <jws>` firmado con la misma clave.

Diagrama (mTLS-bound)

```
AT = { iss, sub, exp, scope, cnf: { "x5t#S256": "AB12..." } }
```

↑
SHA-256(client TLS cert)

RS valida:

- firma AT
- `SHA-256(cliente_TLS_cert_actual) = cnf.x5t#S256`

6.2 ¿Obligatorio?

Sí. FAPI 2.0 §5.3.2.1 establece que `access_token` **MUST be sender-constrained**.

6.3 Soporte Keycloak

✓ Soportado para ambos mecanismos.

mTLS-bound: Clients → Advanced → OAuth 2.0 Mutual TLS Certificate Bound Access Tokens: ON.

DPoP-bound: Clients → Advanced → OAuth 2.0 DPoP Bound Access Tokens: ON.

6.4 Recomendaciones

- En SFA Chile la guía es **mTLS-bound** para B2B (entre instituciones) y **DPoP** para apps móviles donde mTLS es operativamente complejo.
- Verificar en el *Resource Server* (APIs) la cláusula `cnf` antes de aceptar el token; Keycloak Adapters (o `keycloak-quarkus-oidc`) lo hacen, pero APIs custom deben implementarlo.

7. DPoP (Demonstrating Proof-of-Possession)

7.1 Contexto técnico

DPoP (RFC 9449) es una alternativa a mTLS para clientes que **no pueden establecer mTLS** (típicamente SPA, apps móviles, IoT). El cliente:

1. Genera un par de llaves (EC P-256 o RSA).
2. En cada request al AS o RS firma un JWT (*DPoP proof*) con:
 - `htm`: método HTTP (POST).
 - `htu`: URL completa.
 - `iat`: timestamp.
 - `jti`: nonce único.
 - `ath`: SHA-256 del `access_token` (en request al RS).
3. El AS calcula `jkt = SHA-256(JWK(public))` y lo embebe en `cnf.jkt` del `access_token`.
4. El RS valida que `cnf.jkt = SHA-256(JWK del DPoP proof)`.

Ejemplo de DPoP proof header

```
DPoP: eyJ0eXAiOiJkcG9wK2p3dCI6ImFsZyI6IkdVMTJmU2IiwiaWdrIjp7Imt0eSI6IkdVMTIiwiaCI6Iml82c1JKaFRfaWwXcW1nLi4iLCJ5IjojUWtIVHRWQjkuLiIsImNydiI6IiIAtMjU2In19.eyJodHU0IjodHRwczovL2FzL3Rva2VuIiwiaHRtIjojUE9TVCI6ImIhdCI6MTcxNzAwMDAwMCI6ImIwIjojYjYjMTIzIn0.<sig>
```

7.2 ¿Obligatorio?

Condicional. FAPI 2.0 acepta mTLS o DPoP. Se considera obligatorio "uno de los dos". SFA Chile menciona que para clientes con app móvil pública, **DPoP es la vía obligatoria**.

7.3 Soporte Keycloak

✓ Desde Keycloak 24 como *preview*, GA en 26.x.

Habilitar:

```
kc.sh start --features=dpop
```

Por cliente:

```
Clients → <client> → Advanced → DPoP Bound Access Tokens: ON  
→ DPoP nonces: enabled (opcional, anti-replay reforzado)
```

7.4 Recomendaciones

- Activar **DPoP Nonces** (`Use-DPoP-Nonce`) ⇒ obliga al cliente a incluir `nonce` provisto por el AS/RS, mitiga pre-computación.
- Configurar tiempo de tolerancia (`dpop-iat-leeway`) $\leq 60s$.
- En clientes móviles, persistir la clave DPoP en *secure enclave* (iOS Keychain / Android Keystore con `setUserAuthenticationRequired(true)`).

8. DCR (Dynamic Client Registration)

8.1 Contexto técnico

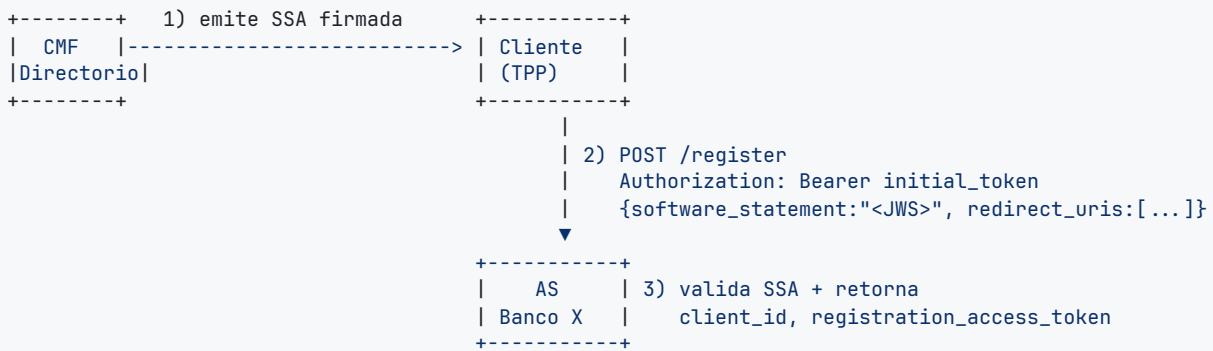
DCR (RFC 7591/7592) permite que un cliente OAuth se registre **programáticamente** en el AS, recibiendo `client_id` / `client_secret` (o `client_certificate`) sin intervención manual.

En Open Finance, DCR es la única vía: el "Directorio" del SFA emite **SSA** firmadas que el cliente entrega al AS de cada Institución Financiera; el AS valida el SSA y crea el registro.

Endpoints

- `POST /register` (RFC 7591): registro inicial.
- `GET/PUT/DELETE /register/{client_id}` (RFC 7592): mantenimiento, autenticado con `registration_access_token`.

Diagrama



8.2 ¿Obligatorio?

Sí para participar del SFA Chile.

8.3 Soporte Keycloak

✓ Parcial.

Keycloak soporta DCR RFC 7591/7592 con:

```

Realm Settings → Client Registration → Initial access tokens
                                     → Trusted hosts
                                     → Policies (validators)

```

Pero **no valida nativamente el software_statement SSA** según las claves públicas del Directorio del SFA y sus claims específicos. Para eso se requiere implementar un **ClientRegistrationPolicy SPI**.

8.4 Recomendaciones (cuando soporte parcial)

Implementar un SPI personalizado:

```

public class SfaSoftwareStatementPolicy implements ClientRegistrationPolicy {
    public void beforeRegister(ClientRegistrationContext ctx) {
        String ssaJws = ctx.getClient().getSoftwareStatement();
        JWSInput input = new JWSInput(ssaJws);
        JWKSet directoryJwks = fetchJwks("https://directorio.cmf.cl/jwks");
        if (!RSAProvider.verify(input, directoryJwks)) throw new ClientRegistrationException("SSA inválido");
        JsonNode claims = JsonSerialization.readValue(input.getContent(), JsonNode.class);
        if (!claims.get("software_environment").asText().equals("production")) throw ...;
        // Copiar org_id, software_id, redirect_uris desde el SSA a los attrs del cliente
    }
}

```

Empaquetar como JAR en `/opt/keycloak/providers/` y registrar:

```
META-INF/services/org.keycloak.services.clientregistration.policy.ClientRegistrationPolicyFactory
```

Y activar la policy en Realm → Client Registration → Policies.

9. Consumo de Software Statements (SSA)

9.1 Contexto técnico

Un **Software Statement Assertion** es un JWT firmado por una autoridad central (en Chile: el **Directorio del SFA / CMF**) que **declara** la identidad y atributos de un *software* cliente:

```
{
  "iss": "https://directorio.cmf.cl",
  "iat": 1717000000,
  "exp": 1748536000,
  "jti": "ssa-9d8e...",
  "org_id": "12345-6",
  "org_name": "Banco Ejemplo S.A.",
  "software_id": "tpp-app-v1",
  "software_client_name": "MiBancoApp",
  "software_jwks_uri": "https://tpp.example.cl/jwks",
  "software_redirect_uris": ["https://tpp.example.cl/cb"],
  "software_roles": ["AISP", "PISP"],
  "software_environment": "production"
}
```

El AS lo recibe en el endpoint DCR (`software_statement`) y lo valida:

1. Firma JWS con clave del Directorio (publicada en `/jwks`).
2. `iss` corresponde al Directorio confiable.
3. `exp` no vencido.
4. `org_id` / `org_status` no esté en lista de revocación.

9.2 ¿Obligatorio?

Sí para SFA Chile.

9.3 Soporte Keycloak

✗ No nativo.

Keycloak conoce el parámetro `software_statement` y lo guarda como atributo, pero **no valida su firma ni claims** contra el Directorio. Se requiere extensión.

9.4 Recomendaciones

- Implementar el SPI de la sección 8.4 con validación completa.
 - Cachear el JWKS del Directorio con $TTL \leq 1$ h.
 - Implementar verificación contra la **lista de organizaciones revocadas** (endpoint del Directorio del SFA).
 - Persistir el `jti` del SSA para evitar replay (`software_statement` reusado).
-

10. JARM (JWT Secured Authorization Response Mode)

10.1 Contexto técnico

En OAuth tradicional, la respuesta de autorización (`code` , `state`) viaja como query parameter en el redirect:

```
https://client/cb?code=xyz&state=abc
```

Esto es manipulable y, peor, *non-repudiable* (no firmado). **JARM** lo reemplaza por un **JWT firmado** y entregado como un único parámetro `response` :

```
https://client/cb?response=eyJhbGciOiJIUzI1NiIsImtpZCI6Ii4uIn0...
```

El JWT contiene:

```
{
  "iss": "https://as.banco.cl",
  "aud": "client-123",
  "exp": 1717000600,
  "code": "xyz",
  "state": "abc"
}
```

Modos: `response_mode=query.jwt` | `fragment.jwt` | `form_post.jwt` | `jwt` .

10.2 ¿Obligatorio?

Obligatorio en FAPI 2.0 Message Signing. Recomendado en Security Profile. Para flujos de pago en SFA Chile, **obligatorio**.

10.3 Soporte Keycloak

☒ Soportado desde Keycloak 21 (preview), GA en 24.x.

Habilitar:

```
kc.sh start --features=jarm
```

Y por cliente:

```
Clients → <client> → Advanced → Authorization Signed Response Alg: PS256
                                   → Authorization Response Mode: jwt
```

10.4 Recomendaciones

- Combinar JARM con **PAR** para firmar tanto el request como la response (cierre end-to-end de la cadena).
- Usar `form_post.jwt` cuando el JWT pueda exceder el límite de URL (~2KB).

11. PAR Obligatorio (Pushed Authorization Requests)

11.1 Contexto técnico

PAR (RFC 9126) elimina el envío de parámetros sensibles vía *front-channel* (URL del navegador). El cliente hace un `POST` al endpoint `/par` autenticado, el AS guarda los parámetros y devuelve un `request_uri` opaco:

```
1) POST /par (mTLS)      2) Redirect /authorize?client_id=..&request_uri=urn:ietf:params:oauth:request_uri
  client_id=...           ↓
  redirect_uri=...        AS recupera la request guardada por request_uri
  scope=...               ↓
  code_challenge=...      [Auth + Consent + redirect a callback]
  response_type=code
  state=...
  → 201
  { "request_uri": "urn:..abc",
    "expires_in": 60 }
```

Beneficios:

- Imposible alterar parámetros vía URL (CSRF de autorización).
- Front-channel sólo lleva `client_id` + `request_uri` opacos.
- Permite *request objects* (JAR) grandes sin chocar con límites de URL.

11.2 ¿Obligatorio?

Sí. FAPI 2.0 §5.3.1.1: *AS MUST require PAR for all authorization requests.*

11.3 Soporte Keycloak

✅ Soportado nativamente.

Habilitar feature `par`. Por defecto el endpoint queda en `/realms/{realm}/protocol/openid-connect/ext/par/request`.

Realm Settings → OpenID Endpoint Configuration → `"pushed_authorization_request_endpoint"`

Por cliente / vía profile:

`secure-par-executor`: (lo activa `fapi-2-security-profile`)

11.4 Recomendaciones

- En el cliente, **siempre** firmar el contenido como `request_object` (JAR) además del POST, para garantizar integridad incluso si el `request_uri` se filtrara.
 - TTL de `request_uri` $\leq 90s$ (recomendación FAPI WG).
-

12. PKCE Obligatorio (S256)

12.1 Contexto técnico

PKCE (RFC 7636) protege el *authorization code* contra interceptación:

1. Cliente genera `code_verifier` (43-128 chars random).
2. `code_challenge` = `BASE64URL(SHA-256(code_verifier))`.
3. Envía `code_challenge` + `code_challenge_method=S256` en la request de autorización.
4. Al canjear el code, envía `code_verifier`.
5. AS comprueba `SHA-256(verifier) = challenge`.

```
code_verifier      = "dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWF0EjXk"
code_challenge_S256= "E9MeIhoa20wvFrEMTJguCHaoeK1t8URWbuGJSstw-cM"
```

FAPI 2.0 obliga **únicamente** S256 (no plain).

12.2 ¿Obligatorio?

Sí.

12.3 Soporte Keycloak

✅ Soportado nativamente.

Por cliente:

```
Clients → Advanced → Proof Key for Code Exchange Code Challenge Method: S256
```

El profile FAPI 2 Security activa el executor `pkce-enforcer-executor`.

12.4 Recomendaciones

- Validar que `code_verifier` tenga **mínimo 43 caracteres** (la RFC lo exige).
 - Verificar que aplicaciones móviles **no** registren el `code_verifier` en logs ni en *URL-schemes intent* (Android intent-filter sin verificación).
-

13. OCSP (Online Certificate Status Protocol)

13.1 Contexto técnico

OCSP (RFC 6960) permite consultar en tiempo real el estado de un certificado X.509 (Good / Revoked / Unknown) contra el responder de la CA:



La respuesta es un `BasicOCSPResponse` firmado por la CA, contiene `thisUpdate`, `nextUpdate`, `certStatus`.

En FAPI/Open Finance se valida el cert presentado por el cliente vía mTLS contra OCSP **en cada conexión** (o cacheado por `nextUpdate`).

13.2 ¿Obligatorio?

Sí. La cadena PKI del SFA exige verificar revocación; OCSP (preferentemente) o CRL como fallback.

13.3 Soporte Keycloak

⚠ Parcial.

Keycloak hace validación de revocación mediante el **X.509 Client Certificate Authenticator** (`x509-direct-grant-authenticator` / `x509-browser-authenticator`):

```
Authentication → Flows → X.509 Direct Grant → Config
CRL Checking Enabled : ON
OCSP Checking Enabled : ON
OCSP Responder URI    : https://ocsp.directorio.cmf.cl
OCSP Responder Cert   : (X.509 PEM)
```

Limitación: la validación OCSP de Keycloak es sobre el cert presentado a su *X509 Authenticator* (login con cert), **no** automáticamente sobre cada token-endpoint mTLS — eso depende del *cert-lookup provider* y la JVM. Se recomienda configurar a nivel de JVM:

```
-Dcom.sun.security.enableCRLDP=true
-Dcom.sun.net.ssl.checkRevocation=true
-Docsp.enable=true
-Docsp.responderURL=https://ocsp.directorio.cmf.cl
```

13.4 Recomendaciones

- Implementar un `X509ClientCertificateAuthenticator` **SPI custom** que, además de la validación PKIX, llame al responder OCSP del SFA, valide su firma con el cert del responder, y aplique caché con

`nextUpdate` .

- Métricas Prometheus de tasa de fallas OCSF para alertar cuando responder esté caído (decidir *fail-open* vs *fail-closed* según política).

14. OCSF Stapling

14.1 Contexto técnico

En vez de que el AS (verificador) consulte OCSF, es el **servidor TLS** quien adjunta (`staples`) la respuesta OCSF firmada en el handshake (TLS Certificate Status Request, RFC 6066). Beneficios:

- Reduce latencia (no hay round-trip extra).
- Mejora privacidad (la CA no ve qué clientes consultan al servidor).
- Funciona aunque el OCSF esté momentáneamente offline (TTL).

Variante: **OCSF Must-Staple** (RFC 7633) — el cert tiene una extensión que **obliga** al server a staplear; si falta, el cliente rechaza.

14.2 ¿Obligatorio?

Recomendado. SFA Chile lo exige *para el certificado del propio AS* (server-side stapling); para el lado *cliente* (cert del TPP), OCSF "tradicional" sigue siendo lo normal.

14.3 Soporte Keycloak

⚠ Vía reverse-proxy.

El stack Quarkus de Keycloak no expone configuración directa de OCSF stapling. La práctica habitual es:

- Terminar TLS en **NGINX**:

```
ssl_stapling on;
ssl_stapling_verify on;
resolver 8.8.8.8 valid=60s;
ssl_trusted_certificate /etc/nginx/ca-chain.pem;
```

- O en **HAProxy** (`set ssl ocsf-response`).
- O en **Envoy** (`ocsf_staple_policy: must_staple`).

14.4 Recomendaciones

- Refrescar la respuesta OCSF cada 4 h (los stapled responses suelen tener TTL de 7 días).
 - Monitorear que `ssl_stapling` esté efectivamente activo (no todos los responders lo soportan; usar `openssl s_client -status -connect ...`).
-

15. Consentimiento Granular e Informativo

15.1 Contexto técnico

El consentimiento debe ser:

- **Granular:** cada *scope* / cada *recurso legal* aprobado individualmente.
- **Informativo:** explicación clara al usuario de qué dato se compartirá, con quién, por cuánto tiempo, para qué fin.
- **Auditable:** trazabilidad con identificador único (`consent_id`), fecha, hash de los términos mostrados.
- **Revocable:** en cualquier momento por el usuario, propagando revocación a tokens activos.

Ejemplo SFA: un usuario otorga acceso a:

- Cuentas (`accounts:read`) por 12 meses.
- Saldo (`balances:read`) por 12 meses.
- Movimientos últimos 90 días (`transactions:read?from=2025-08-25&to=2025-11-25`).
- **Pero NO** acepta `direct-debits:write` .

15.2 ¿Obligatorio?

Sí. SFA Chile exige consentimiento explícito por *recurso* y por *propósito*.

15.3 Soporte Keycloak

⚠ Parcial.

Keycloak tiene una *consent screen* configurable (`Client Scopes` → `consent screen text`) pero:

- No soporta nativamente parámetros dinámicos (rango de fechas, cuentas específicas).
- No persiste un `consent_id` enriquecido (solo lista de scopes aceptados por cliente).
- Revocación: existe `RevocationEndpoint` para tokens, y "consent revoke" en Account Console, pero sin propagar a un servicio externo.

15.4 Recomendaciones

Patrón recomendado en Open Finance (similar a Brasil):

1. **Pre-creación de consentimiento** fuera de Keycloak: el TPP llama a un *Consent API* del banco que crea un `consent_id` con detalles completos.
2. El TPP pasa el `consent_id` en `authorization_details` (RAR – ver §16) o en un scope dinámico (`consent:<consent_id>`).
3. Keycloak renderiza la pantalla de consentimiento delegando en una **Authenticator SPI** que consulta el Consent API y muestra los datos enriquecidos al usuario.
4. Al aceptar, se asocia el `consent_id` al token (claim `consent_id`).
5. Revocación: actualizar Consent API + invocar `RevocationEndpoint` de Keycloak para los tokens vivos.

16. Rich Authorization Requests (RAR – RFC 9396)

16.1 Contexto técnico

RAR introduce un nuevo parámetro `authorization_details` (JSON) que reemplaza al limitado `scope` :

```
"authorization_details": [
  {
    "type": "payment_initiation",
    "locations": ["https://api.banco.cl/payments"],
    "instructedAmount": {"currency": "CLP", "amount": "150000"},
    "creditorAccount": {"iban": "CL0000123..."},
    "creditorName": "Juan Pérez"
  },
  {
    "type": "account_information",
    "locations": ["https://api.banco.cl/accounts"],
    "accounts": ["CL0099887..."],
    "permissions": ["ReadBalances", "ReadTransactionsBasic"]
  }
]
```

Permite expresar *exactamente* qué se autoriza (no sólo "leer cuentas" sino "leer esta cuenta específica, sólo balances, sólo este día").

16.2 ¿Obligatorio?

Opcional en FAPI 2.0 base, obligatorio en SFA Chile para *payment initiation* y otros casos de uso de escritura.

16.3 Soporte Keycloak

⚠ Parcial.

Keycloak puede transportar `authorization_details` como claim en el `access_token` (con un *Protocol Mapper* tipo *Hardcoded/Script*), pero **no lo procesa semánticamente** (no valida `type`, no lo muestra en `consent screen` estructurado).

16.4 Recomendaciones

- Implementar un `AuthorizationDetailsValidator` **SPI** que:
 - Reconozca los `type` válidos definidos por el SFA Chile (`payment_initiation`, `account_information`, etc.).
 - Valide el esquema JSON de cada tipo.
 - Inyecte el contenido en `id_token` y `access_token` como claim `authorization_details`.

- Pantalla de consent que renderice los `authorization_details` en lenguaje humano ("Vas a autorizar el pago de \$150.000 a Juan Pérez desde tu cuenta XYZ").

17. Mapeo de Scopes y Recursos Legales

17.1 Contexto técnico

Cada *scope* OAuth o *type* RAR debe estar **mapeado** a:

- Recurso legal (Ej: "Ley 21.521 art. 16 letra c – datos de cuentas").
- Finalidad declarada al usuario.
- TTL máximo permitido por norma.
- Roles del cliente (AISP / PISP) habilitados.

Tabla típica:

Scope	Recurso Legal	Roles	TTL máx	Sensibilidad
<code>accounts:read</code>	Art. 16(c) NCG 514	AISP	365d	Media
<code>balances:read</code>	Art. 16(c) NCG 514	AISP	365d	Media
<code>transactions:read</code>	Art. 16(d) NCG 514	AISP	90d	Alta
<code>payments:write</code>	Art. 17 NCG 514	PISP	1d (single)	Crítica

17.2 ¿Obligatorio?

Sí, como parte del cumplimiento normativo y de la documentación de evaluación de impacto en datos personales (Ley 19.628 reformulada).

17.3 Soporte Keycloak

✗ No soportado nativamente.

Keycloak conoce *Client Scopes* y permite metadata por scope, pero no expone modelos "recurso legal", "finalidad", "TTL por scope".

17.4 Recomendaciones

- Implementar un **catálogo central** (DB o configmap) de scopes ↔ recurso legal, mantenido por el equipo de Compliance.
- En el `Authenticator` que muestra el consent screen, leer el catálogo y desplegar el detalle (cumple además §15).
- En el `AccessTokenMapper`, embeber `legal_basis` y `purpose` como claims (útil para auditoría aguas abajo).
- En CI/CD, validar que todo scope nuevo en Keycloak tenga su entrada en el catálogo.

18.1 Contexto técnico

- No puedan ser modificados a posteriori sin detección (integridad).
- La autoría sea verificable (autenticidad).
- Idealmente, encadenados (hash chain estilo blockchain ligero) para detectar eliminaciones.

a) **Logs de verificación de certificados (mTLS / OCSP):** por cada handshake mTLS, registrar `client_cert_sha256`, `cn`, `serial`, `ocsp_status`, `ocsp_response_signature_valid`, `timestamp`.

b) **Logs de validación de SSA:** por cada DCR, registrar `ssa_jti`, `ssa_iss`, `org_id`, `validation_result`, `directory_jwks_kid`, `timestamp`.

```
{
  "event_id": "01H...",
  "event_type": "mtls_handshake",
  "ts": "2026-05-25T16:00:00Z",
  "actor": "client-123",
  "data": { "client_cert_sha256": "AB12...", "ocsp_status": "good" },
  "prev_hash": "f3a9...", // encadenamiento
  "self_hash": "7c2d...",
  "signature": "eyJhbGciOiJIUzI1NiIsImtpZCI6Imt..." // JWS sobre {event sin signature}
}
```

18.3 Soporte Keycloak

- No los firma criptográficamente.
- No encadena hashes.
- No persiste eventos de bajo nivel como handshake mTLS (esos viven en el reverse-proxy o JVM).

18.4 Recomendaciones

Patrón propuesto:

1. Captura:

- Eventos Keycloak → `EventListenerProvider` SPI custom.
- Eventos mTLS → módulo de log de NGINX/Envoy enviando a Kafka.
- Eventos OCSP/SSA → desde los SPI propios (§13, §9).

2. Firma + cadena: servicio `log-signer` (microservicio) que:

- Recibe eventos vía Kafka topic `audit.raw`.
- Calcula `prev_hash` (consulta último evento por partition).
- Firma con clave en HSM (PS256).
- Publica en topic `audit.signed`.

3. Almacenamiento WORM: `audit.signed` → S3 con *Object Lock* compliance mode (o equivalente Object Storage on-prem) + replica a *time-stamping authority* TSA (RFC 3161) para sello de tiempo confiable.

4. Verificación: comando CLI / batch nocturno que recorre cadena y verifica firmas y hashes; alerta SIEM si rompe.

Stack sugerido: Keycloak SPI → Kafka → Flink/Quarkus signer → S3 (Object Lock) → ELK para indexación + Grafana para visualización.
